

AIR QUALITY PREDICTOR (AQI)

BY

MUHAMMAD JAMSHAD



**A TECHNICAL REPORT SUBMITTED IN FULFILLMENT
OF REQUIREMENT FOR THE PROJECT**

10PEARL SHINE INTERNSHIP

FOR

DATA SCIENCE

10PEARLS

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to 10Pearls for the opportunity to undertake this Data Science Internship and to work on a real, end-to-end MLOps problem rather than an isolated notebook exercise. I am especially thankful to my project reviewers, whose detailed technical feedback on the initial submission — pointing out that the 3-day forecast was not genuinely model-driven, that the training horizon did not match the forecast requirement, and that the live and historical data sources did not match — directly shaped the redesigned system documented in this report. Diagnosing and correcting those issues, along with a number of platform-level failures encountered along the way, was the most valuable part of this internship experience.

ABSTRACT

Air pollution is one of the most pressing environmental and public-health concerns facing rapidly urbanising cities, and Lahore, Pakistan, consistently ranks among the most polluted cities in the world. This technical report introduces the Lahore AQI Predictor, an end-to-end, serverless Air Quality Index (AQI) prediction system developed as part of the 10Pearls Data Science Internship. The system ingests hourly pollutant and meteorological data from the Open-Meteo public API, engineers and cleans a rich feature set, stores it in a Hopsworks Feature Store, trains and evaluates five machine learning algorithms — Random Forest, Gradient Boosting, Ridge Regression, Lasso Regression, and a feed-forward Neural Network — across four distinct forecasting horizons (next-hour, +24h, +48h, and +72h), registers the resulting models to a Hopsworks Model Registry, and serves live predictions and a genuine 3-day forecast through an interactive Streamlit dashboard with SHAP-based model explainability, kept continuously up to date through two scheduled GitHub Actions pipelines.

A significant part of this project's engineering effort was spent diagnosing and correcting an initial implementation that produced the appearance of a working forecast without any of the underlying machine learning actually driving it — a next-hour-only model was in production, while the displayed "3-day forecast" was generated from a random sine-wave perturbation of the last observed value. This report documents that defect and twelve other concrete technical issues found and fixed during development, together with the evidence and reasoning behind each fix, and closes with an honest evaluation of model performance per horizon, a discussion of current system limitations, and recommendations for future work.

Table of Contents

CHAPTER 1	3
INTRODUCTION	3
1.1 Background	3
1.2 Description	3
1.3 Problem Statement	3
1.4 Scope	4
1.5 Objectives	4
1.6 Feasibility Study	5
CHAPTER 2	15
SOFTWARE REQUIREMENT SPECIFICATION (SRS)	15
2.1 System Overview and Problem Background	15
2.2 Purpose of the System	15
2.3 Scope of the System	15
2.4 Overall Description	15
2.5 Functional Requirements	16
2.6 Non-Functional Requirements	16
2.7 Performance Requirements	17
2.8 Reliability and Availability	17
2.9 Hardware Requirements	17
2.10 Software Requirements	17
2.11 System Constraints	18
2.12 Future Enhancements	18
2.13 Conclusion	18
CHAPTER 3	19
SYSTEM ANALYSIS	19
3.1 Problem Identification	19
3.2 Existing System Analysis	19
3.3 Proposed System Overview	20
3.4 System Objectives	20
3.5–3.6 Functional and Non-Functional Requirements	20
3.7 System Architecture	20
3.8 Data Flow Analysis	21
3.9 Use Case Analysis	21
3.10–3.11 Hardware and Software Requirements	22
3.12 Future Enhancements	22

3.13 Summary	22
CHAPTER 4	23
SYSTEM DESIGN AND ARCHITECTURE	23
4.1 Module Design.....	23
Module	23
Responsibility	23
backfill_data.py.....	23
CHAPTER 5	32
TESTING, EVALUATION, AND CONCLUSION	32
5.1 System Verification, Validation and Quality Assurance Framework.....	32
5.2 Comprehensive Testing Strategy, Planning and Execution Methodology	32
5.3 Functional Behavior Analysis Under Diverse Input Conditions	32
5.4 System Performance Evaluation and Computational Efficiency Assessment	33
5.5 Accuracy Evaluation, Precision Analysis and Model Reliability Study.....	33
5.6 Comparative Analysis with the Earlier (Pre-Weather) System	35
5.7 Error Handling Mechanisms, System Stability and Exception Management.....	36
5.8 Dataset Evaluation, Training Impact and Data Dependency Analysis	36
5.9 System Reliability Testing and Long-Term Performance Observation.....	37
5.10 Final Project Outcome Analysis and Practical Implementation Results	37
5.11 Future Enhancement Possibilities and Expansion Opportunities	37
5.12 Overall Project Performance Summary and Technical Achievement Review ..	38
CONCLUSION.....	39
PROJECT GITHUB LINK AND QR-CODE.....	40

CHAPTER 1

INTRODUCTION

1.1 Background

The Air Quality Index (AQI) is a standardised measure, published by regulatory bodies such as the US Environmental Protection Agency (EPA), that translates raw pollutant concentrations — most importantly fine particulate matter (PM_{2.5}) — into a single number on a 0–500 scale, together with an associated health category (Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, Hazardous). Lahore regularly experiences AQI readings well into the "Unhealthy" and "Hazardous" categories, particularly during the autumn and winter smog season, making reliable short-term forecasting a matter of genuine public value: residents, schools, and outdoor workers can make better decisions if they know not only today's air quality but also a credible estimate of tomorrow's and the day after's.

1.2 Description

The Lahore AQI Predictor is a specialised, end-to-end MLOps platform built to forecast air quality for Lahore across multiple time horizons. The system ingests hourly pollutant and weather observations from the Open-Meteo API, engineers and cleans a feature set describing the current atmospheric state, stores it in a Hopsworks Feature Store, trains five different regression algorithms against four independent forecast horizons, registers every model together with its evaluation metrics to a Hopsworks Model Registry, and serves live predictions, a real 3-day forecast, health-risk alerts, and SHAP-based explainability through an interactive Streamlit dashboard. Two GitHub Actions workflows keep the system continuously current without manual intervention: one refreshes the feature store hourly, and one retrains and re-registers every model daily.

1.3 Problem Statement

Given hourly historical pollutant and weather observations for Lahore, the objective is to (a) predict air quality one hour ahead with high accuracy for immediate situational awareness, and (b) produce a genuinely model-driven 3-day-ahead forecast (at +24h, +48h, and +72h) that a resident could use for forward planning — all served through a live,

continuously-updating web dashboard, with the underlying models retrained on a daily cadence as new data becomes available. An initial version of this system appeared to satisfy this requirement but did not: its "3-day forecast" was a mathematically fabricated curve with no connection to any trained model, which this project's core engineering work set out to fix.

1.4 Scope

The scope of this project covers the city of Lahore, Pakistan, using publicly available data from the Open-Meteo Air Quality and Weather APIs. The system is intentionally built on free-tier, serverless infrastructure — Hopsworks for the Feature Store and Model Registry, GitHub Actions for orchestration, and Streamlit for the dashboard — to keep it reproducible and cost-free. This choice also introduced a number of platform-level constraints and failure modes that are documented in Chapter 4.

1.5 Objectives

- Build an automated feature pipeline that ingests hourly pollutant and weather data, cleans it, and stores it in a centralised feature store.
- Engineer a feature set including time-based features, lag features, rolling statistics, and change-rate indicators.
- Train and rigorously evaluate a diverse set of models — spanning classical linear models, tree ensembles, and a neural network — using RMSE, MAE, R^2 , and MAPE.
- Design a genuine multi-horizon forecasting capability (next-hour, +24h, +48h, +72h) rather than a single next-step model presented as a multi-day forecast.
- Register all trained models to a model registry with full versioning and metrics, and have the serving layer automatically select the best-performing, most recent model per horizon.
- Serve live predictions and the 3-day forecast through a dashboard, converted to the true AQI scale and accompanied by health-risk alerts.
- Provide model explainability using SHAP so predictions are not a black box.
- Automate the entire feature-refresh and retraining cycle using CI/CD (GitHub Actions).

1.6 Feasibility Study

The project is technically feasible using entirely free-tier services: Open-Meteo's public API requires no API key or payment, Hopsworks offers a free-tier Feature Store and Model Registry sufficient for this workload, GitHub Actions provides free scheduled compute for public repositories, and Streamlit Community Cloud can host the dashboard at no cost. The required skill set — Python, pandas, scikit-learn, TensorFlow/Keras, and REST API integration — was within scope for the internship timeline. The main feasibility risk that materialised during development was not technical skill but platform reliability: several Hopsworks-side failures (documented in Section 4.6) consumed substantial debugging time and had to be worked around rather than "fixed" outright, since they originated outside this project's own code.

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 System Overview and Problem Background

The Lahore AQI Predictor addresses the absence of a genuinely model-driven, multi-day air-quality forecast for Lahore. Existing public sources (e.g. IQAir, AQICN) report current and short-term forecast AQI but do not expose an open, retrainable, explainable pipeline; this project builds that pipeline end-to-end, from raw data ingestion through to a live, explainable dashboard.

2.2 Purpose of the System

The purpose of the system is to automatically ingest air-quality and weather data for Lahore on an hourly basis, maintain a clean and versioned feature history, train and evaluate multiple forecasting models across four horizons, and present live status, a 3-day forecast, and model explanations to an end user through a web dashboard — all without manual intervention once deployed.

2.3 Scope of the System

In scope: hourly data ingestion (Open-Meteo), data cleaning, feature engineering, model training across four horizons and five algorithms, model registration and versioned selection, a Streamlit dashboard with live AQI, 3-day forecast, health alerts and SHAP explanations, and full CI/CD automation. Out of scope: coverage of cities other than Lahore, mobile push notifications, user accounts/authentication, and use of an actual future weather forecast (rather than current atmospheric state) as a model input — the last of these is discussed as future work in Section 5.11.

2.4 Overall Description

The system is composed of four cooperating components: (1) a live feature pipeline that runs hourly, (2) a backfill script that populates historical training data, (3) a daily training pipeline that trains and registers models, and (4) a Streamlit web application that serves predictions and visualisations. All four share the Hopsworks Feature Store and Model Registry as their single source of truth, so that training and serving are always working from identical feature definitions.

2.5 Functional Requirements

1. The system shall fetch hourly pollutant (PM2.5, PM10, O3) and weather (temperature, humidity, wind, pressure, precipitation, cloud cover) data from Open-Meteo.
2. The system shall clean the raw data (duplicate removal, missing-value imputation, outlier capping) before it is used for training.
3. The system shall engineer time-based, lag, and rolling-mean features, and compute four forecast-horizon targets (next-hour, +24h, +48h, +72h).
4. The system shall store engineered features in a versioned Hopworks Feature Store with an explicit schema.
5. The system shall train five algorithms per horizon and evaluate each with RMSE, MAE, R^2 , and MAPE.
6. The system shall register every trained model, together with its metrics, to the Hopworks Model Registry.
7. The system shall automatically select, at serving time, the best-performing model among the most recently trained batch for each horizon.
8. The system shall convert raw PM2.5 model output into the true US EPA AQI index before display.
9. The system shall display a live dashboard showing current AQI, next-hour prediction, a 3-day forecast, a health-risk alert, a historical trend chart, and SHAP-based explanations.
10. The system shall automatically refresh features hourly and retrain models daily without manual intervention.

2.6 Non-Functional Requirements

- Reliability: the serving layer must gracefully degrade (fall back to the next-best model) rather than crash if a specific registry entry is corrupted or unavailable.
- Maintainability: feature engineering logic must be defined once and reused identically by both the backfill and live pipelines, to prevent training/serving skew.
- Explainability: predictions must be accompanied by a model-agnostic explanation (SHAP), regardless of which underlying algorithm is currently active.
- Automation: the system must operate continuously via scheduled CI/CD without requiring a human to manually re-run any pipeline.
- Cost: the entire system must run on free-tier infrastructure.

2.7 Performance Requirements

The live feature pipeline is expected to complete within a few minutes per hourly run. The daily training pipeline, which trains and registers 20 models (5 algorithms $\times$ 4 horizons), is expected to complete within roughly 90 minutes under normal conditions, and a GitHub Actions timeout of 90 minutes is configured accordingly. The Streamlit dashboard is expected to load and render live predictions within a few seconds under cached conditions.

2.8 Reliability and Availability

Because the system depends on a third-party, free-tier platform (Hopsworks) for both the Feature Store and Model Registry, availability is bounded by that platform's own reliability. Several transient Hopsworks backend failures were observed and are documented in Section 4.6; the system's resilience strategy is to retry transient failures automatically and to degrade gracefully (serving the next-best available model) rather than to guarantee zero downtime, which is outside this project's control.

2.9 Hardware Requirements

No dedicated hardware is required. Training runs on GitHub Actions' standard hosted runners (CPU-only); the Neural Network model is small enough (three hidden layers of 128/64/32 units) that CPU training completes in a reasonable time without requiring GPU acceleration.

2.10 Software Requirements

Component	Requirement
Language	Python 3.12
ML libraries	scikit-learn, TensorFlow/Keras
Explainability	SHAP, Matplotlib
Data libraries	Pandas, NumPy, Requests
Feature Store / Model Registry	Hopsworks SDK
Dashboard	Streamlit, Plotly
CI/CD	GitHub Actions

2.11 System Constraints

- All infrastructure must remain within free-tier limits (Hopsworks, GitHub Actions).
- Open-Meteo's exact historical data-availability window is undocumented, so the backfill process must degrade gracefully across multiple window sizes.
- The Hopsworks client library in use (v4.0+) no longer supports Hive-based reads, constraining the retry strategy for feature-group reads to plain Arrow-Flight retries.

2.12 Future Enhancements

See Section 5.11 for a full discussion of future work, including incorporating actual weather forecasts, sequence models, and multi-city expansion.

2.13 Conclusion

This chapter has specified the functional and non-functional requirements the system must satisfy. Chapter 3 analyses how the initial implementation failed to meet several of these requirements and how the proposed, corrected system addresses them.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Problem Identification

A structured code review of the initial implementation, conducted as part of this internship's review process, identified that the dashboard's "3-day forecast" function accepted a trained model, a scaler, and a feature list as parameters but never referenced any of them in its body — it instead computed $\text{last_value} + 8 \cdot \sin((\text{hour}-6) \cdot \pi/12) + \text{Gaussian_noise}$, a fabricated diurnal curve. This was confirmed by the observable symptom that the forecast changed on every page reload, which is inconsistent with a deterministic model prediction. Three related problems were identified alongside it: the only trained target was a next-hour target (`pm25_avg.shift(-1)`), so no model existed that had ever learned a genuine 24/48/72-hour-ahead pattern; the live feature pipeline ingested from a different provider (AQICN, daily resolution) than the historical backfill (Open-Meteo, hourly resolution), creating a train/serve distribution mismatch; and `inference_pipeline.py`, the module that should have contained the real inference logic, was effectively an empty placeholder file.

3.2 Existing System Analysis

Aspect	Existing (Flawed) System
3-day forecast	Sine-wave + random noise; not connected to any model
Training horizon	Next-hour only (<code>shift(-1)</code>)
Live data source	AQICN (daily resolution)
Backfill data source	Open-Meteo (hourly resolution)
<code>inference_pipeline.py</code>	Empty placeholder
Model selection	Single hard-coded model, no version/RMSE comparison

3.3 Proposed System Overview

Aspect	Proposed (Corrected) System
3-day forecast	Real predictions from dedicated +24h/+48h/+72h models
Training horizon	Four horizons: next-hour, +24h, +48h, +72h, each with its own direct target
Live data source	Open-Meteo (hourly resolution)
Backfill data source	Open-Meteo (hourly resolution) — identical source and schema
<code>inference_pipeline.py</code>	Full implementation: loads the best model per horizon and predicts from the latest real feature row
Model selection	Filters to the latest trained batch (max version), then picks lowest test RMSE, with graceful fallback

3.4 System Objectives

Restated from Section 1.5, refined against the identified defects: replace every simulated or mismatched component with a genuinely data-driven, source-consistent, and automatically-selected equivalent, without weakening any already-correct part of the pipeline.

3.5–3.6 Functional and Non-Functional Requirements

Unchanged from Sections 2.5 and 2.6; System Analysis confirmed that the existing system failed functional requirements 1 (data source consistency), 3 (multi-horizon targets), 7 (best-model selection), and 9 (real forecast display), while requirements 2, 4, 5, 6, 8, and 10 were substantially met.

3.7 System Architecture

The corrected architecture is illustrated below and detailed further in Chapter 4:

Open-Meteo API → Hourly Feature Pipeline → Hopsworks Feature Store → Daily Training Pipeline → Hopsworks Model Registry → Inference Layer + Streamlit Dashboard

3.8 Data Flow Analysis

11. Open-Meteo Air Quality API and Weather API are queried for pollutant and weather observations, respectively.
12. `backfill_data.py` merges the two responses on their shared hourly timestamp and computes 25 engineered features plus 4 horizon targets.
13. `data_cleaning.py` further processes the resulting file: fixing data types, removing duplicates, imputing missing values via linear interpolation and median fill, capping outliers with the IQR method, dropping zero-variance columns, and rounding numeric values.
14. `uploading_to_hopsworks.py` creates or evolves the Hopsworks feature-group schema and uploads the cleaned dataset.
15. `feature_pipeline.py` repeats steps 1–2 for a small recent window every hour, reusing the same functions, and upserts directly into the same feature group.
16. `training_pipeline.py` reads the full feature history, trains 5 algorithms $\times$ 4 horizons, and registers every model with its evaluation metrics.
17. `app.py` (via `inference_pipeline.py`) reads the latest feature row and the model registry, selects the best model per horizon, predicts, converts to the true AQI scale, and renders the dashboard.

3.9 Use Case Analysis

Actor	Use Case
End user (resident)	View current AQI, next-hour prediction, and 3-day forecast; view health-risk alert
Data scientist / maintainer	Inspect SHAP explanations and per-algorithm metrics to judge model trustworthiness

Actor	Use Case
CI/CD scheduler (GitHub Actions)	Trigger the hourly feature refresh and the daily training/registration run automatically
Developer	Manually trigger a pipeline (workflow_dispatch) or inspect raw Feature Store data via the dashboard's data tab

3.10–3.11 Hardware and Software Requirements

Unchanged from Sections 2.9 and 2.10.

3.12 Future Enhancements

See Section 5.11.

3.13 Summary

This chapter traced each defect identified in the existing system to a specific, concrete correction in the proposed system, and outlined the corrected data flow. Chapter 4 describes the design of each corrected module in technical detail, followed by a full account of the engineering challenges encountered while implementing it.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 Module Design

Module	Responsibility
backfill_data.py	One-time/periodic historical population: fetches pollutant + weather data from Open-Meteo, merges, engineers 25 features and 4 targets
data_cleaning.py	Post-processes the raw backfill CSV: type fixing, deduplication, missing-value imputation, IQR outlier capping, zero-variance column removal, rounding
uploading_to_hopsworks.py	Creates/evolves the Hopsworks feature-group schema and uploads the cleaned dataset
feature_pipeline.py	Hourly live refresh: reuses backfill_data.py's own functions on a small recent window and upserts directly to Hopsworks
training_pipeline.py	Trains 5 algorithms $\times$ 4 horizons, evaluates each, and registers all resulting models to the Hopsworks Model Registry
inference_pipeline.py	Loads the best model per horizon from the registry and predicts from the latest real feature row
app.py	Streamlit dashboard: live AQI, 3-day forecast, health alerts, historical trend chart, SHAP explanations, raw data browser

4.2 Data Design

The Hopsworks feature group `aqi_predictions` (v7) is keyed on (city, date, hour), with timestamp as the event-time column, using the HUDI time-travel format. Its schema comprises:

Category	Fields
Keys / time	city, date, timestamp, hour, day, month, day_of_week, is_weekend
Pollutants	o3_avg, o3_max, pm10_avg, pm10_max, pm10_min, pm25_avg, pm25_max, pm25_min, uvi_avg, uvi_max
Weather	temperature_2m, relative_humidity_2m, wind_speed_10m, wind_direction_10m, surface_pressure, precipitation, cloud_cover
Derived	aqi_lag_1, aqi_change_rate, aqi_rolling_mean_3
Targets	aqi_target, aqi_target_24h, aqi_target_48h, aqi_target_72h

The schema was extended twice after its initial creation — once for the three additional forecast-horizon targets and once for the seven weather columns — using Hopsworks' schema-evolution API (`append_features`) rather than creating a new feature-group version each time, keeping historical rows and the live pipeline on a single, continuously-evolving schema.

The `data_cleaning.py` module applies a dedicated cleaning stage between raw backfill and upload: it coerces datetime and numeric columns explicitly, removes exact and (city, date, hour)-key duplicates, fills numeric gaps by time-ordered linear interpolation followed by median fill, winsorizes outliers using $1.5 \times \text{IQR}$ bounds (capping rather than dropping, to preserve every training timestamp), drops any column with zero variance unless it is a protected calendar/derived/sensor field, defers categorical one-hot encoding to training time to keep the Feature Store schema stable, and rounds all numeric columns to two decimal places before writing the final cleaned CSV.

4.3 Model Design

Four forecast horizons are trained, each against its own direct ground-truth target obtained by shifting the hourly PM2.5 series forward in time — not by chaining a single next-step model's own output back into itself recursively:

Horizon key	Target column	Meaning
next_hour	aqi_target	PM2.5 one hour ahead
day1_24h	aqi_target_24h	PM2.5 twenty-four hours ahead
day2_48h	aqi_target_48h	PM2.5 forty-eight hours ahead
day3_72h	aqi_target_72h	PM2.5 seventy-two hours ahead

Five algorithms are trained for every horizon:

- Ridge Regression — L2-regularised linear model on standardised features.
- Lasso Regression — L1-regularised linear model on standardised features.
- Random Forest Regressor — 100 estimators, max depth 20.
- Gradient Boosting Regressor — 100 estimators, max depth 5, learning rate 0.1.
- Feed-forward Neural Network — Dense(128, ReLU) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.2) → Dense(32, ReLU) → Dense(1), Adam optimiser, MSE loss, early stopping (patience 15), on standardised features.

At serving time, for every horizon, the dashboard queries the registry for all five algorithm variants, filters to only the most recently trained batch (the maximum registered version number for that horizon), and among that batch selects the model with the lowest test RMSE — falling back to the next-best model if a specific registry entry's weights cannot be downloaded or loaded.

4.4 Explainability Design

Because the serving layer can select any one of five structurally different algorithms as the "best" model for a given horizon at any point in time, a model-specific explainability approach would be inconsistent across algorithm switches. The dashboard therefore uses SHAP via a single, model-agnostic wrapper: a `predict_fn(X)` function that applies the correct scaler (if any) and then calls the underlying model's own `.predict()`, regardless of whether that model is a Random Forest, a linear model, or a Keras neural network. `shap.Explainer` is given this callable directly, letting SHAP automatically select an appropriate algorithm (Permutation explainer) rather than requiring a different SHAP explainer class per model family. Results are shown as a mean-|SHAP-value| bar chart and a beeswarm plot, which additionally conveys the direction of each feature's effect.

4.5 Software Testing Strategy

Given the absence of a formal staging environment, this project's testing strategy was primarily diagnostic and empirical: every pipeline stage was run against the live Hopsworks project and its real output (terminal logs, Hopsworks UI state, and dashboard screenshots) was inspected to confirm or refute a hypothesis about a defect, rather than relying on isolated unit tests against mocked infrastructure. Where a defect's cause was ambiguous, standalone isolation scripts were written to test one hypothesis at a time — for example, a script that registered a completely fresh, never-used model name to rule out a naming-related cause for a persistent registration failure (Section 4.6.4), and a script that force-downloaded and attempted to load a specific model's artefacts outside the dashboard to isolate a Keras deserialisation error from the surrounding application code (Section 4.6.6). The final docx report generation was itself verified by round-tripping the produced file through both `python-docx` and LibreOffice before being considered complete, after an early draft was found to contain a structurally invalid XML element.

4.6 Engineering Discussion: Challenges Encountered and Their Solutions

This section documents, in the order they were discovered, the concrete technical defects found during development and how each was diagnosed and resolved. It is presented in detail deliberately: this process was a substantial part of the learning outcome of the internship, and several of these issues (4.6.1, 4.6.3, and 4.6.9 in particular) were raised directly by project reviewers and required genuine redesign rather than a superficial patch.

4.6.1 Simulated 3-Day Forecast (Critical Reviewer Finding)

Issue: The dashboard displayed a "3-day forecast", but `generate_3day_forecast()` never called the trained model. It computed `last_value + 8·sin((hour-6)·π/12) + Gaussian noise` — a fixed diurnal sine wave plus randomness. Because the noise term was re-sampled on every page load, the forecast visibly changed on every refresh, which was the observable symptom that exposed the defect.

Root Cause: The function signature accepted `model`, `scaler`, and `feature_names` as parameters, giving the appearance of being model-driven, but none of the three were referenced anywhere inside the function body.

Solution: `inference_pipeline.py` was implemented from scratch to load the best registered model for each of the three forecast horizons and predict directly from the latest real feature row, and `app.py` was changed to import and call this real implementation. The fabricated function was deleted entirely.

4.6.2 Next-Step-Only Training vs. a Multi-Day Forecast Requirement

Issue: The only trained target was `pm25_avg.shift(-1)` (one hour ahead); no target anywhere in the pipeline corresponded to a genuine 24/48/72-hour-ahead value.

Root Cause: The original pipeline was designed purely around next-step prediction and was never extended when the 3-day forecast requirement was added to the dashboard.

Solution: Three additional direct targets — `aqi_target_24h/48h/72h` — were added via `shift(-24)/-48/-72` on the hourly series, and `training_pipeline.py` was restructured to loop over four horizons, training and registering five algorithms for each.

4.6.3 Mismatched Data Sources Between Backfill and Live Pipeline

Issue: Historical training data was backfilled from Open-Meteo at hourly resolution, while the live feature pipeline ingested from AQICN's daily forecast endpoint — a different provider at a different resolution, feeding the same downstream schema.

Root Cause: The two pipelines were written independently and never reconciled, so the model was trained on one statistical distribution and served on another.

Solution: The AQICN-based live pipeline was removed entirely. `feature_pipeline.py` now calls the same Open-Meteo functions used by `backfill_data.py`, requesting a small recent window instead of a large historical one, guaranteeing training and serving data always share the same source, fields, and resolution.

4.6.4 Hopsworks Model Registry Instability During Registration

Issue: Several model-registration calls failed with transient server-side errors: HTTP 500 responses containing a `ClusterJDatastoreException` ("Tuple did not exist") from Hopsworks' NDB cluster, and separately, connection-reset errors after the client had already been "waiting for model registration" for 17–38 minutes.

Root Cause: Isolation testing (registering a brand-new, never-used model name, with an explicit API key, after several delayed retries, with a minimal payload) reproduced the identical failure, ruling out a client-side or naming-related cause and confirming the issue was on Hopsworks' backend.

Solution: A dedicated retry script was written to re-register only the specific models that failed, reusing the already-trained local artefacts. The inference layer was also made resilient to this class of failure by trying every algorithm variant for a horizon, best RMSE first, falling back if a registry entry is missing or unreadable.

4.6.5 Corrupted / Empty Model Artefacts in the Registry

Issue: Several registered models showed correct metadata and metrics in the Hopsworks UI, but their "Browse model files" view was empty.

Root Cause: A downstream symptom of the same platform instability in 4.6.4: model metadata creation and file upload are two separate steps, and failures were occurring specifically in the second step.

Solution: Missing files were manually re-uploaded via the Hopsworks file browser as an immediate workaround, and the inference layer's fallback logic was hardened to skip any model whose expected weights file is absent, trying the next-best algorithm instead of crashing the dashboard.

4.6.6 Keras Legacy HDF5 Deserialization Failure

Issue: Loading a saved neural-network model with `load_model()` raised `ValueError: Could not deserialize 'keras.metrics.mse' because it is not a KerasSaveable subclass`.

Root Cause: A known Keras 3.x regression when loading the legacy .h5 format: the loader attempts to deserialize the model's full compiled training configuration, and that path is broken for models saved under an earlier Keras/TensorFlow version.

Solution: Since the model is only needed for inference, it is loaded with `load_model(h5_path, compile=False)`, which skips the broken training-configuration deserialization while still loading weights and architecture correctly.

4.6.7 Missing Scaler for the Neural Network Model

Issue: The neural network was trained on standardised features, exactly like Ridge and Lasso, but the model-registration code only wrote `scaler.pkl` for the ridge and lasso branches, not for `neural_network`.

Root Cause: An omission in the per-algorithm branch of the model-saving logic in `training_pipeline.py`.

Solution: Because a single shared `StandardScaler` instance is fit once and reused across the ridge, lasso, and neural-network branches, an existing `scaler.pkl` could be copied directly into the neural-network model's registry folder as an immediate fix, and the training script was corrected to always save the scaler alongside the neural-network artefact going forward.

4.6.8 Weak Predictive Performance on the 24h/48h/72h Horizons

Issue: Initial 24h/48h/72h models produced negative R^2 (worse than predicting the historical mean), and the resulting dashboard forecast cards showed suspiciously similar values across all three horizons.

Root Cause: A model with near-zero or negative R^2 has not learned a genuine predictive signal and regresses toward the training-set mean; a 90-day backfill window gave too little seasonal variation, and the feature set contained no atmospheric-state signal (wind, pressure, humidity) to help anticipate dispersion/build-up.

Solution: The backfill window was extended (with an automatic 730/500/365-day fallback), and seven weather features were added. This measurably improved every horizon (Chapter 5), though the multi-day horizons remain a substantially harder problem than next-hour prediction and are documented as an ongoing limitation.

4.6.9 Raw PM2.5 Concentration Displayed as if it Were the AQI Index

Issue: The dashboard's "Predicted AQI" values (e.g. ~55) were far lower than AQI figures reported by independent public sources (e.g. IQAir reporting ~116 for the same city and time).

Root Cause: Every trained target, despite its name, is a raw PM2.5 concentration in $\mu\text{g}/\text{m}^3$ — no EPA breakpoint conversion had ever been applied anywhere in the pipeline.

Solution: A `pm25_to_aqi()` function implementing the official US EPA piecewise-linear breakpoint formula was added and applied at every point a value is presented as "AQI" — the KPIs, the three forecast cards, the delta calculation, and the chart — while "Current PM2.5 Level" was deliberately left as a raw $\mu\text{g}/\text{m}^3$ figure, since that label is accurate as-is.

4.6.10 Dependency Conflict Between SHAP and scikit-learn

Issue: After installing shap, the dashboard failed to load any Gradient Boosting model with No module named 'sklearn.ensemble._gb_losses'.

Root Cause: Installing shap pulled in a newer scikit-learn as a transitive dependency; the newer version reorganised its internal loss-function modules, so already-pickled models could no longer be deserialized.

Solution: scikit-learn was pinned to a pre-1.3 version in requirements.txt, compatible with the already-trained artefacts and preventing any future pip install from silently upgrading it again.

4.6.11 Missing GitHub Actions Secret Broke the Automated Feature Pipeline

Issue: The first live run of the hourly workflow failed with ExternalClientError: host cannot be of type NoneType.

Root Cause: `hopsworks.login()` was called with `host=os.getenv("HOPSWORKS_HOST")`, but that repository secret had never been created — only `HOPSWORKS_API_KEY`, `HOPSWORKS_PROJECT`, and the now-unused `AQICN_API_KEY` existed.

Solution: The `HOPSWORKS_HOST` secret was added under Repository Settings, and both workflow files were confirmed to reference all three required secrets before being re-triggered successfully.

4.6.12 Broken Hive Read Fallback on Hopsworks Client v4.0+

Issue: A defensive fallback to Hopsworks' Hive read engine, added earlier for Arrow Flight failures, began raising its own hard failure: Reading data with Hive is not supported when using hopsworks client version ≥ 4.0 .

Root Cause: Hive-based reads were removed from newer Hopsworks client versions, so the fallback — written against an older client's capabilities — was now guaranteed to fail on every invocation, masking the original, usually-transient Arrow Flight error.

Solution: The Hive fallback was removed entirely and replaced with a simple retry-with-delay (up to three attempts, ten seconds apart) around the plain feature-group read, compatible with the current client and still able to recover from transient Arrow Flight errors.

4.6.13 Streamlit's `st.stop()` Not Reliably Halting Execution

Issue: Early in development, a `NameError` occurred inside `app.py` even though an `st.stop()` call appeared, in the source, immediately after the exception that should have prevented execution from continuing.

Root Cause: The application was being launched with `python app.py` rather than `streamlit run app.py` — outside Streamlit's script-runner context, `st.stop()` does not behave as documented.

Solution: Beyond correcting the launch command, the code was made robust to this class of failure by initialising `model/df` to safe defaults before any Hopsworks call and wrapping dependent dashboard logic in an explicit `if model is None or df.empty: ... else: ...` block.

CHAPTER 5

TESTING, EVALUATION, AND CONCLUSION

5.1 System Verification, Validation and Quality Assurance Framework

Verification of this system was carried out against three criteria: (a) does each pipeline complete without an unhandled exception when run against the live Hopsworks project; (b) does the value displayed on the dashboard trace back to an actual model prediction rather than a placeholder or simulated value; and (c) does the displayed AQI number fall within a plausible range of independently-reported AQI for the same city and time. Criterion (b) is the direct validation for the central defect described in Section 4.6.1, and was checked by confirming the forecast no longer changed between reloads with an unchanged feature store.

5.2 Comprehensive Testing Strategy, Planning and Execution Methodology

Testing proceeded pipeline-by-pipeline, in dependency order: the feature pipeline was validated first (confirming correct row counts and schema in Hopsworks), then the training pipeline (confirming all 20 models registered with plausible metrics), then the inference layer and dashboard (confirming displayed values matched what the registry actually contained). Each stage's terminal output and the corresponding Hopsworks UI state were cross-checked before moving to the next stage, since several defects (Sections 4.6.4 and 4.6.5) were only visible in the UI, not in the client library's own success/failure return value.

5.3 Functional Behavior Analysis Under Diverse Input Conditions

The system was exercised under several non-nominal conditions during development: a feature group with missing weather columns (verified to fill safely with 0.0 rather than crash, per the guard in `transform_and_align_schema`); a model registry with some model names entirely absent (verified to raise a clear error listing which candidate names were checked); and a registry entry present but missing its weights file (verified to be skipped in favour of the next-best model rather than crashing the dashboard).

5.4 System Performance Evaluation and Computational Efficiency Assessment

The hourly feature pipeline completes in well under the 15-minute GitHub Actions timeout configured for it. The daily training pipeline, training and registering 20 models against roughly 8,800 hourly rows, has been observed to take between 20 minutes (all registrations succeeding promptly) and over 90 minutes (when Hopsworks-side registration delays of the kind described in Section 4.6.4 occur); the workflow timeout is set to 90 minutes accordingly, with a separate standalone retry script available to re-register any individual model that fails without repeating the full run.

5.5 Accuracy Evaluation, Precision Analysis and Model Reliability Study

The most recent fully-reviewed training run used 365 days of hourly backfill data (8,784 rows) enriched with seven weather features, split 80/20 chronologically (7,027 train / 1,757 test rows per horizon).

Horizon	Best Algorithm	Test RMSE	Test R ²	Test MAPE
Next-hour	Lasso	9.37	0.8926	13.73%
+24h (Day 1)	Neural Network	28.66	0.0263	38.51%
+48h (Day 2)	Gradient Boosting	31.63	-0.1411	37.42%
+72h (Day 3)	Random Forest	31.54	-0.0875	37.80%

Full per-algorithm results, next-hour horizon:

Model	MAE	RMSE	R ²	MAPE
Lasso	6.46	9.37	0.8926	13.73%

Model	MAE	RMSE	R ²	MAPE
Ridge	6.49	9.44	0.8910	13.43%
Neural Network	7.30	9.87	0.8808	13.64%
Gradient Boosting	5.20	10.09	0.8755	8.35%
Random Forest	5.15	10.20	0.8727	8.16%

Full per-algorithm results, +24h horizon:

Model	MAE	RMSE	R ²	MAPE
Neural Network	20.65	28.66	0.0263	38.51%
Gradient Boosting	20.12	28.70	0.0238	35.79%
Random Forest	20.47	29.09	-0.0028	36.59%
Lasso	21.40	30.03	-0.0691	39.67%
Ridge	21.51	30.13	-0.0757	39.91%

Full per-algorithm results, +48h horizon:

Model	MAE	RMSE	R ²	MAPE
Gradient Boosting	22.26	31.63	-0.1411	37.42%
Random Forest	22.47	31.71	-0.1472	38.66%

Model	MAE	RMSE	R ²	MAPE
Neural Network	24.77	33.61	-0.2890	42.76%
Lasso	25.36	35.39	-0.4288	43.79%
Ridge	25.52	35.54	-0.4410	44.15%

Full per-algorithm results, +72h horizon:

Model	MAE	RMSE	R ²	MAPE
Random Forest	22.46	31.54	-0.0875	37.80%
Gradient Boosting	22.35	31.64	-0.0945	36.14%
Neural Network	24.69	34.05	-0.2676	40.86%
Lasso	26.01	36.37	-0.4457	42.51%
Ridge	26.10	36.47	-0.4537	42.66%

5.6 Comparative Analysis with the Earlier (Pre-Weather) System

The table below compares this run against an earlier run using only 90 days of backfill data and no weather features:

Horizon	R ² — 90 days, no weather	R ² — 365 days + weather	Change
Next-hour	0.9556	0.8926	-0.063 (slightly worse)
+24h	-0.3509	0.0263	+0.377 (large improvement)

Horizon	R^2 — 90 days, no weather	R^2 — 365 days + weather	Change
+48h	-1.2071	-0.1411	+1.066 (large improvement)
+72h	-1.1486	-0.0875	+1.061 (large improvement)

Extending history and adding weather features substantially improved every multi-day horizon, though none reached strong predictive territory. The slight regression in next-hour R^2 is most likely attributable to the larger dataset including more genuine year-round variation than the smaller, single-season 90-day window; absolute next-hour accuracy ($RMSE \approx 9.4$) remains strong. A further backfill extension toward roughly two years of history was subsequently applied to the pipeline; a final, reviewed comparison table for that specific run had not yet been produced at the time of writing and is called out explicitly in Section 5.11.

5.7 Error Handling Mechanisms, System Stability and Exception Management

Every external call to Hopsworks (feature-group reads, model downloads, model registration) is wrapped in exception handling that either retries (feature-group reads, up to three attempts with a delay) or falls back to an alternative (model loading, trying the next-best algorithm variant). The dashboard's top-level model/feature-loading logic is guarded so that a failure to load either one is confined to a single if/else branch, and no downstream code path can execute against an undefined model or dataframe (Section 4.6.13).

5.8 Dataset Evaluation, Training Impact and Data Dependency Analysis

The 90-day-to-365-day comparison in Section 5.6 is direct empirical evidence that the multi-day horizons are substantially data-limited: the same features, algorithms, and code produced markedly better R^2 purely from more historical rows and additional weather covariates. This indicates the current ceiling on 24h/48h/72h accuracy is more a function of available training signal than of algorithm choice, which directly motivates the future-

work priorities in Section 5.11 (more data, and a genuine future-weather input rather than only the current atmospheric state).

5.9 System Reliability Testing and Long-Term Performance Observation

Ongoing reliability is provided by two scheduled GitHub Actions workflows: an hourly feature refresh and a daily retraining run, both authenticated via repository secrets rather than committed credentials, and both manually triggerable for on-demand testing. Both workflows were verified to run successfully after the missing-secret defect in Section 4.6.11 was corrected.

5.10 Final Project Outcome Analysis and Practical Implementation Results

The final system satisfies every functional requirement in Section 2.5: it ingests and cleans real hourly data, trains and registers 20 genuinely-differentiated models across four horizons, automatically selects the best current model per horizon, converts predictions to the true AQI scale, and serves the result — alongside a real 3-day forecast and SHAP explanations — through a live, continuously-updating dashboard. Every component the initial review identified as simulated, mismatched, or missing (Section 3.2) now has a concrete, verified, working replacement (Section 3.3).

5.11 Future Enhancement Possibilities and Expansion Opportunities

- Complete and formally review the ~2-year backfill experiment referenced in Section 5.6.
- Incorporate actual future weather forecasts (rather than only current atmospheric state) as model inputs for the multi-day horizons — the single change most likely to materially improve 24h–72h accuracy.
- Explore sequence models (LSTM, GRU, Temporal Fusion Transformer) as an alternative or complement to the current per-horizon tabular-feature approach.
- Add prediction intervals or quantile estimates so the dashboard can honestly communicate the wide uncertainty in the multi-day forecasts.
- Add automated model-performance monitoring and data-drift detection to the daily training pipeline.

- Extend the system to additional Pakistani cities using the same architecture, parameterised by city.
- Add push notifications or a subscription mechanism for hazardous-AQI alerts.
- Conduct a systematic hyperparameter search (e.g. via Optuna) per algorithm/horizon combination.
- Migrate neural-network artefacts to Keras's native .keras format once tooling support stabilises, retiring the legacy-HDF5 `compile=False` workaround.

5.12 Overall Project Performance Summary and Technical Achievement Review

This project achieved strong next-hour forecasting accuracy ($R^2 \approx 0.89$) and a genuine, if still limited, multi-day predictive signal (R^2 near zero but consistently improved by additional data and features), while replacing every simulated or mismatched component identified during review with a verified, working implementation, fully automated via CI/CD and explained via SHAP. The process of correctly diagnosing thirteen distinct technical issues — several of which were platform-level failures rather than bugs in this project's own code — constituted a significant part of the practical MLOps learning this internship was designed to provide.

CONCLUSION

The Lahore AQI Predictor set out to build a complete, automated, and genuinely model-driven AQI forecasting system, and — after a substantial amount of debugging and redesign documented in this report — achieves that goal. Every stage of the pipeline is now backed by a genuine, auditable implementation rather than any shortcut or simulated behaviour, is fully automated via GitHub Actions, and is explainable via SHAP. Next-hour forecasts are accurate and reliable; the 3-day forecast is real and model-driven, though its predictive power remains limited and is documented honestly rather than overstated. The engineering journey recorded in Chapter 4 — diagnosing a fabricated forecast, a data-source mismatch, a missing multi-horizon training design, and a series of platform-level Hopsworks failures — represents the core practical learning outcome of this internship

PROJECT GITHUB LINK AND QR-CODE



[https://github.com/ mjamshaid786/aqi-predictor](https://github.com/mjamshaid786/aqi-predictor)